SJÄLVSTÄNDIGA ARBETEN I MATEMATIK

MATEMATISKA INSTITUTIONEN, STOCKHOLMS UNIVERSITET

Gröbner Bases and Elimination in Macaulay 2

av

Christian Eriksson

2026 - No 5

Gröbner Bases and Elimination in Macaulay 2

Christian Eriksson

Självständigt arbete i matematik 15 högskolepoäng, grundnivå

Handledare: Sofia Tirabassi

2026

Abstract

Contents

1 Introduction

Something goes here

2 The Algebraic Abstraction of Geometry

Algebra is the mathematical expression of the human need for structure and abstraction. Through algebra, humans try to capture more of the mathematical, and natural, domain under their pen. The endless hunger for knowledge can express itself in various way. When using algebra as a tool, creating abstract models that capture generic structure is favored. However, these abstract models can be difficult to understand. The ability to navigate between levels of abstraction is a vital tool for any mathematician interested in applying algebraic methods.

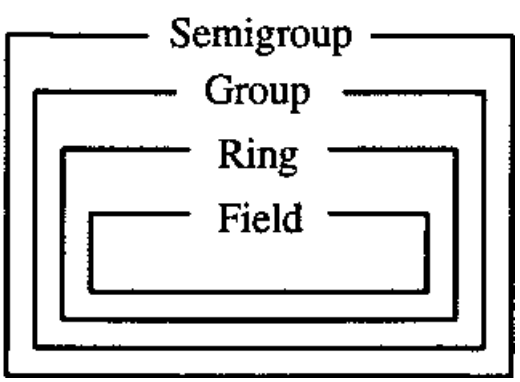
This paper assumes a degree of familiarity with algebraic structures, but the uninitiated peruser is kindly directed towards the easy-going introduction of "Introduction to Modern Algebra" [Wei70] with bite size exercises, or alternatively, the more rigorous "Abstract Algebra" [DF03] for a deeper understanding.

As the section header suggests, this section will be primarily focused with defining the tools necessary to build an analytic bridge between geometry and algebra so that we can analyze geometric objects with algebra. A function that maps geometric objects to algebraic must be established. This function uses Affine Varieties and Ideals, along with the dictionary established in the Hilbert Strong Nullstellensatz, to accomplish that goal.

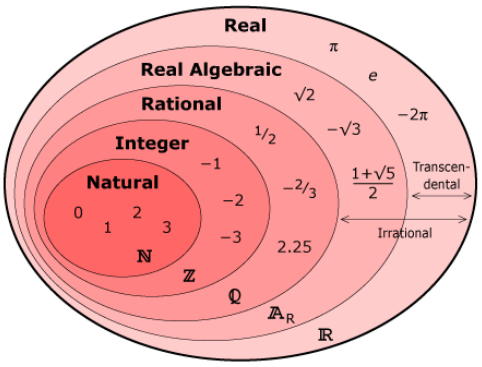
2.1 Levels of Abstraction

Abstract algebra traverses through layers of abstraction as an exercise. As an example, if we are interested in how groups act when supplemented with commutivity (Abelian groups), it would be useful to start at groups and traverse the layers of figure 1a downwards. Note that climbing upwards may not be as fruitful for this particular case.

Choosing an Abelian group that stays between groups and rings is $(\mathbb{R}, +)$. Here the set of real numbers is only considered with the binary operator $+$, and commutivity follows. Analyzing this group and all the properties that are associated with it is of interest. However, much use can be extracted from climbing further down the abstraction ladder and analyzing rings a. Keeping it concrete with numbers, one can consider the abelian ring $\mathbb{Z}$. Time can be spent on this layer of abstraction and knowledge can be obtained through leveraging $\mathbb{Z}$'s special properties. Going even further down the ladder, to a field, one could analyze $\mathbb{Q}$. At each layer, new intuition can be gained. Note also, that while traversing downwards in the group abstraction,



(a) Abstractions within Groups [Lat]



(b) Generalization of Numbers [Diand]

Figure 1: Traversing the ladder of Abstraction

we were climbing up and down in the generalization of numbers. Starting at the top with $\mathbb{R}$, we jumped all the way down to $\mathbb{Z}$, and then up again to $\mathbb{Q}$. It is important to consider this, as different layers of abstraction do not necessarily have to correspond with another. Caution is required when dealing with this iterative abstract process.

2.2 Polynomials

In order to address the problem at hand fully, we need to generalize the idea of a polynomial. This is done by compressing more and more information into a single mathematical expression. A process of abstraction is necessary. We start with the monomial, continue to a polynomial, and finally consider polynomial rings. All work done is done on these polynomials rings, and this process of abstraction enables us to say something about all monomials simultaneously.

Definition 2.1. A **monomial** in $x_1, \ldots, x_n$ is a product of the form

$$x^\alpha = x_1^{\alpha_1} \cdot x_2^{\alpha_2} \cdots x_n^{\alpha_n} \tag{1}$$

where all $\alpha_1, ..., \alpha_n$ are nonnegative integers.

As the root word "mono" suggests, there is only one of these terms x^α. However, as we are creating the tools to generalize and deal with more abstract concepts, we also want to include the ability to capture any linear combination of these monomials.

Definition 2.2. A **polynomial** f in $x_1, \ldots, x_n$ with coefficients, $a_1, \ldots, a_n$, in a

field $\mathbb{K}$ is a finite linear combination of monomials.

$$f = \sum_{\alpha} a_\alpha x^\alpha, \quad a_\alpha \in \mathbb{K} \tag{2}$$

With the polynomial definitions at hand, we can construct the final abstraction for our mathematics involving polynomials, $\mathbb{K}[x] = \mathbb{K}[x_1, \ldots, x_n]$. An important note is that $\mathbb{K}[x]$ satisfies the axioms for rings and is therefore referred to as the *polynomial ring*. When leveraging this powerful abstraction, we capture n dimensions of variables on any field $\mathbb{K}$. This enables more compact analysis.

2.3 Affine Spaces and Varieties

To start the bridge between the world of algebra as we know it and geometry, there must be an established way to discuss geometry algebraically. This is where the concepts of affine spaces come in.

Also talk about what geometry actually is in this context (points, lines, and functions, and so on).

This concept allows the analyst to view geometry algebraically. However, this is not the end goal, but an analytic step between geometry and the more powerful algebraic tools that we want to apply to geometry.

It is also worth relating how vector spaces and affine spaces use maps.

" The geometric properties of a vector space are invariant under the group of bijective linear maps, whereas the geometric properties of an affine space are invariant under the group of bijective affine maps, and these two groups are not isomorphic." [Gal25]

Affine maps, crudely speaking, are generalizations of linear maps [Gal25]. This generalization can be viewed in different ways, but a few important examples are listed.

1. Affine spaces allow for a dynamic origin. Vector spaces do not.

2. There are more affine maps than there are linear maps.

3. Affine spaces study points in a more general matter, independently of the coordinate system chosen. Frame invariance.

2.3.1 Affine Spaces

In order to understand affine varieties, affine spaces must be understood. This coordinate-agnostic space is of interest since more abstract entities can be modeled upon it. Let us first consider the definition:

Definition 2.3. An *affine space* is either the empty set, or a triple $\langle E, \vec{E}, + \rangle$. E is a nonempty set of points while $\vec{E}$ is a vector space (of translations). The action $+$ is an operation on both E and $\vec{E}$ defined as follows: $+ : E \times \vec{E} \to E$. The following axioms are assumed for affine space:

A1 $a + 0 = a$ for every $a \in E$.

A2 $(a + u) + v = a + (u + v)$ for every $a \in E$ and every $u, v \in \vec{E}$.

A3 For any two points $a, b \in E$, there is a unique $u \in \vec{E}$ such that $a + u = b$

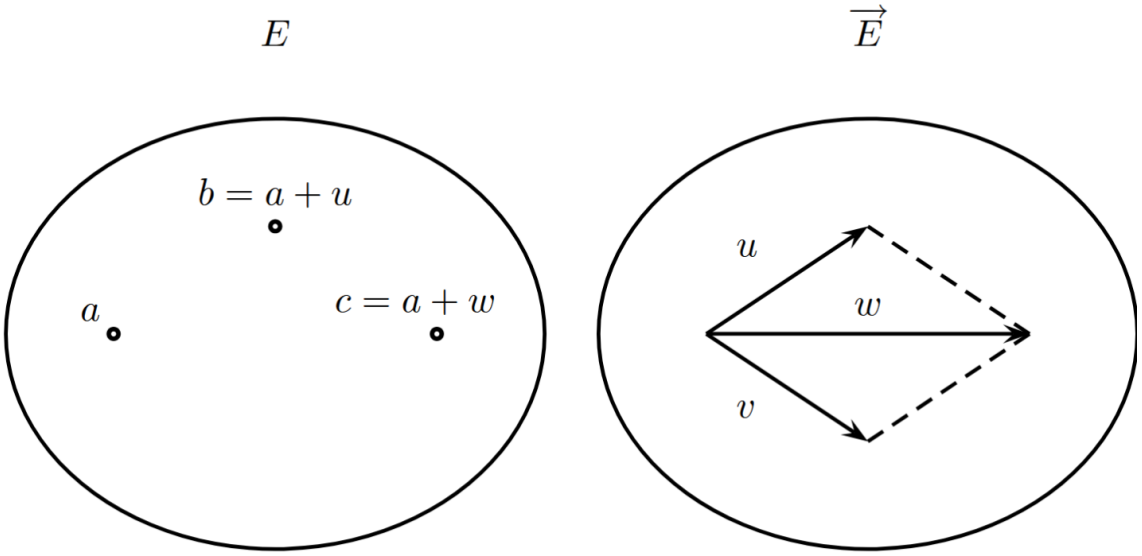


Figure 2: Intuitive understanding of affine space [Gal25]

A quick note of importance on each axiom. If you start at point a and do nothing, you should still be at a. This seems obvious and silly to include, but these are axiomatic assumptions. We must treat them with the respect they deserve. Axiom two deals with the associative property. The fact that the order of operations makes little difference to the outcome is of great interest. To simply calculations algebraically, rearranging order is always of interest. The last axiom, that of uniqueness is also of importance. Being able to assume that an unique element in $\vec{E}$ always

exists that can connect two points a and b is useful when solving these problems on a higher level of abstraction.

The definition is lengthy, but the intuition should be clear when considering Figure 2. Here a separation between the vectors and the points *to which the vector is applied* should be noted. These two can be defined independently. This separation is exactly what enabled the coordinate-agnostic property that is of interest.

A supplemental definition will be provided for the affine to clear up any confusion.

Example 2.4. Given a field $\mathbb{K}$ and a positive integer n, define the n-dimensional *affine space* over $\mathbb{K}$ as:

$$\mathbb{K}^n := \langle \left\{ [\hat{x}_1, \ldots, \hat{x}_n]^T : \hat{x}_i \in \mathbb{K}, i = 1, \ldots, n \right\}, \vec{\mathbb{K}^n}, + \rangle \tag{3}$$

Where $\vec{\mathbb{K}^n}$ is the vector space of translations, and $+$ denotes the action of shifting a point by a vector via component-wise addition.

This supplement elucidates the uncertainty behind how this space is defined, as with all other spaces. Its vector space has translations, and they are restricted to the field upon which they operate.

2.3.2 Polynomials in Affine Spaces

It is time to relate the previously discussed polynomial to affine spaces. Leveraging definition **??**, we can relate the two. [LM21] refers to this as a "key idea" and shows that the polynomial $p \in \mathbb{K}[x]$ yields a polynomial *function* as follows:

$$p : \mathbb{K}^n \to \mathbb{K} \tag{4}$$

The function maps the multidimensional field onto the one-dimensional codomain. This p takes any $\hat{x} \in \mathbb{K}^n$ and associates a unique $p(\hat{x})$ to it. This is done easily by substituting $x = \hat{x}$ in the polynomial p. One important note is relevant to the zeros of p when taking $\mathbb{K}$ as an infinite field (e.g. $\mathbb{Z}, \mathbb{R}$). Then p is the zero polynomial if and only if $p(\hat{x}) = 0$ for all $\hat{x} \in \mathbb{K}^n$.

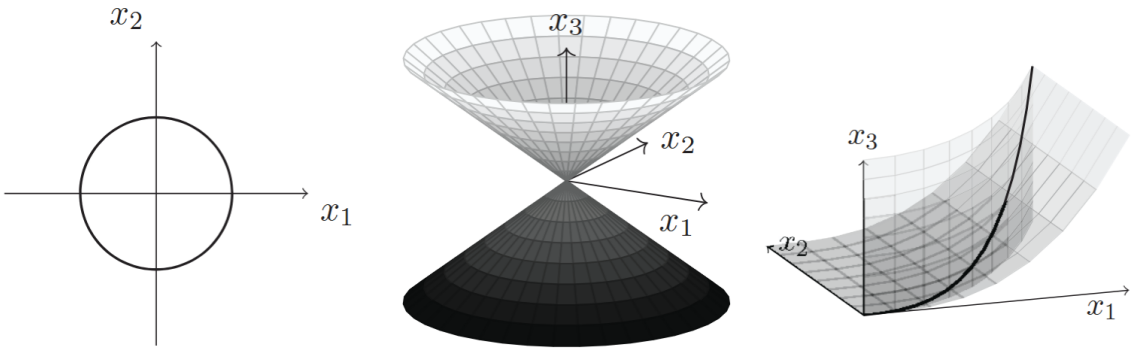
2.3.3 Affine Varieties

With an understanding of how the space we will operate upon will work, the first algebraic structure of geometry will be discussed: affine varieties.

Definition 2.5. Let $\mathbb{K}$ be a field, and let $f_1, \ldots, f_s$ be polynomials in $\mathbb{K}[x_1, \ldots, x_n]$. Then, set

$$\boldsymbol{V}(f_1, \ldots, f_s) = \{(a_1, \ldots, a_n) \in \mathbb{K}^n | \ f_i(a_1, \ldots, a_n) = 0 \ \forall 1 \leq i \leq s\} \tag{5}$$

We call $\boldsymbol{V}(f_1, \ldots, f_s)$ the **affine variety** defined by $f_1, \ldots, f_s$.



(a) $\mathbf{V}(x_1^2 + x_2^2 - 1)$. (b) $\mathbf{V}(x_1^2 + x_2^2 - x_3^2)$. (c) $\mathbf{V}(x_1^2 - x_2, x_1^3 - x_3)$.

Figure 3: Examples of affine spaces [LM21]

In other words, the affine variety is the set of $(a_1, \ldots, a_n)$ that solve all the equations in the variety simultaneously. It is the intersection of all the polynomial roots. Even though $\boldsymbol{V}$ might simply look like a graphed equation, Figure **??**a provides the basis for intuition. While it may look like a regular circular graph, with familiar x and y coordinates, the keen observer will note that we are dealing with two dimensions, x_1, x_2, and that the graph is in fact the set where this polynomial is equal to 0. If we extract the polynomial from the variety and set it to 0, we end up with $x_1^2 + x_2^2 = 1$. Graphing this equation makes the affine variety clear, but it is not necessary. This is especially true when we are interested in n dimensions.

While the above example may make sense, confusion remains as to what the difference between the variety and simply graphing the equation *is*. This difference is made clear by inspecting **??**c. Here, just as in **??**a, we graph the solution of each individual *polynomial* separately. Once we have the individual solutions, we must find their intersection, as that is where the affine variety lies. The difference between an affine variety and simply graphing the solutions to the individual equations becomes clear in higher dimensions.

3 Algebra Theory

With affine varieties under our belt, there is an obvious follow up to this knowledge. Especially for those interested in the real application of this mathematics. Namely, how do we solve these varieties? It is great to describe them, and on a theoretical level it is powerful but to apply them is a challenge in itself. Luckily, the problem of asking for the affine variety points boils down to solving a system of polynomials equations. This is easier said than done, and an issue arises when going to greater and greater dimensions. Most real world problems will involve many dimensions, and being able to solve these problems quickly and with confidence is of importance. To do so, a computer algebra system is most helpful.

Every computer language has important data structures that determine how the language is shaped. For Macaulay 2, ideals is one of these types. They are important to algebra, and native support for manipulating ideals exist. However, to efficiently handle ideals, there is a problem that needs to be solved with great confidence. Namely: how do we go between abstract entities such as ideals and something that a computer can interact with?

3.1 Rings And Ideals

The ideal description problem formulates the bridge between the abstract and machine world. It does this by concretizing the abstract concept of an ideal to a tangible set of generating polynomials that can be interacted with and evaluated. In order to bridge this gap, finite polynomials have to be found. Starting off with monomial ideals in $k[x_1, \ldots, x_n]$ is the best way to do so.

3.1.1 Monomial Ideals

Much like when introducing affine varieties, starting with a single monomial instead of the combination of them reduces complexity. So, when studying the ideals, we start with the same logic of considering a smaller problem. Consider the following definition.

Definition 3.1. An ideal $I \subseteq k[x_1, \ldots, x_n]$ is a monomial ideal that is determined by a set $A \subset \mathbb{Z}^n$. All polynomials in the ideal can be written as a linear combination of $\sum_{\alpha \in A} h_\alpha x^\alpha$ where h_α lies in $k[x_1, \ldots, x_n]$.

A quick tip when dealing with monomial ideals: they can largely be thought of as their exponents. This is their main benefit and why they are important when dealing with computer algebra systems. We will see this property shine in the coming proofs. As an example, consider the following lemma:

Lemma 3.2. *Let $I = \langle x^\alpha | \alpha \in A \rangle$ be a monomial ideal. Then a monomial x^β lies in I if and only if x^β is divisible by x^α for some $\alpha \in A$.*

Proof. This can be proven by considering the exponents. Starting off with showing x^β to x^α, the exponents can be leveraged. If β is a multiple of α then x^β is in the ideal by definition since x^α generates the ideal. The other way around is slightly trickier. Using the definition of the monomial ideal, we can write x^β as the polynomial sum of generators of the monomial ideal and then write h_i as the sum of generators again.

$$x^\beta = \sum_{i=1}^{s} h_i x^{\alpha(i)} = \sum_{i=1}^{s} \left(\sum_j c_{i,j} x^{\beta(i,j)} \right) x^{\alpha(i)} = \sum_{i,j} c_{i,j} x^{\beta(i,j)} x^{\alpha(i)} \tag{6}$$

Here we can see that every term on the right hand side is divisible by x^α, so then x^β on the left hand side must also be so. This proves that we can follow our intuition regarding the exponents and their divisibility much closer. After all, it goes both ways.

□

3.1.2 Dicksons Lemma

This lemma is really the meat of why we are interested in monomial ideals. The ability to reduce ideals to finite generators is the step towards closing the gap between the machine and abstract.

Theorem 3.3. *Let $I = \langle x^\alpha | \alpha \in A \rangle$ be a monomial ideal. Then I can be finitely written by $I = \langle x^{\alpha(1)}, ..., x^{\alpha(s)} \rangle$, where all α lie in A.*

Proof. The proof provided by the book [DAC10] takes an inductive approach. Starting off with $n = 1$ for our $k[x_1, \ldots, x_n]$, it is easy to see that the smallest element of $A \subseteq \mathbb{Z}_{\geq 0}$ generates the remaining elements. When $n > 1$, the interesting part of the proof continues. Namely, we have the monomial ideal, call it J, from the $n-1$ case. This is finite by our induction, so when our A goes from a subset of $\mathbb{Z}_{\geq 0}^{n-1}$ to $\mathbb{Z}_{\geq 0}^{n}$, we can generate slices of these J ideals with the new $x_n = y$ variable. We take some sufficiently large $m \geq 0$ and group the J_l ideals by their corresponding y value

up to $m-1$. In other words, let the ideal J_l be generated by x^β where $x^\beta y^l \in I$. These J_l slices form a "topological map" of the ideal I. Thus I is also finite. $\square$

After a finite ideal is generated, the monomial ideal membership problem is solved. This is since we can take A polynomial and check if the remainder from division by the monomial ideal is 0. If it is, it belongs to the monomial ideal.

3.2 Hilbert Bases Theorem

Here the results that monomial ideals have finite generators is extended to polynomial ideals. The basic idea is that by rearranging the ideal to a monomial ideal using the leading term, we can apply the Dicksons lemma. The first step towards bridging the gap between polynomial and monomial ideals is by crafting an ideal of interest. Namely, the ideal generated by the leading term of a polynomial ideal.

Definition 3.4. Let $I \subseteq k[x_1, \ldots, x_n]$ be an ideal other than the empty on and fix a monomial ordering. Then

1. $LT(I) = \{cx^\alpha \mid \text{there exists } f \in I \backslash \{0\} \text{ with } LT(f) = cx^\alpha\}$.

2. $\langle LT(I) \rangle$ is the ideal generated by the elements of $LT(I)$.

$LT(I)$ is really rather large, and one might question its usefulness. But it is the bridge between general ideals and monomial ideals. After all, the difference between a monomial and the lead term of the polynomial are the other monomials in the polynomial. If we can capture them by constructing an ideal that generates everything by using the leading term, we can create a superset to operate upon. $\langle LT(I) \rangle$ is that set.

Proposition 3.5. *Let $I \subseteq k[x_1, \ldots, x_n]$ be an ideal different from the empty one.*

1. *$\langle LT(I) \rangle$ is a monomial ideal.*

2. *There are $g_1, \ldots, g_t \in I$ such that $\langle LT(I) \rangle = \langle LT(g_1), \ldots, LT(g_t) \rangle$.*

Proof. 1. Since we have used the leading term to construct an ideal, the difference between the monomial ideal and $\langle LT(I) \rangle$ is a nonzero constant. These two equal each other [[DAC10]].

2. Dicksons Lemma tells us that monomial ideals are finitely generated. Since $\langle LT(I)\rangle$ only differs from a monomial ideal by a nonzero constant, Dicksons lemma can be applied here as well. [DAC10].

□

With all this information accumulated, we can formally prove that every polynomial ideal has a finite generating set. This theorem is known as the Hilber Basis Theorem.

Theorem 3.6. *Every Ideal $I \in k[x_1, \ldots, x_n]$ has a finite generating set. In other words, $I = \langle g_1, \ldots, g_t\rangle$ for some $g_1, \ldots, g_t \in I$.*

Proof is given in chapter 2 of Cox [DAC10] as Theorem 4 on page 77.

3.3 Gröbner basis

The generating set $\langle LT(I)\rangle$ was introduced in the last chapter. As we noted then, it is a large set. Dealing with monomials reduces the complexity drastically compared with polynomials, so if we can use $\langle LT(I)\rangle$ instead of the polynomial ideal, it would simplify calculations. Recall that the Hilbert Basis theorem stated that every polynomial ideal has a finite generating set. However, it is not unique. We will start with an ideal that is of special interest to the world of polynomial calculation.

Definition 3.7. Choose a monomial ordering on the polynomial ring $\mathbb{K}[x]$. A finite nonzero subset, $G = \{g_1, \ldots, g_t\}$ of an ideal, I, which is a subset of $\mathbb{K}[x]$ is a **Gröbner Basis** if

$$\langle LT(g_1), ..., LT(g_t)\rangle = \langle LT(I)\rangle \tag{7}$$

The construction is purposeful; we can operate on an object of immense size, $\langle LT(I)\rangle$, while restricting ourselves to a finite number of generators. This makes the theoretical exploration less complex (due to the construction of $\langle LT(I)\rangle$), while the instruction to a computer becomes guaranteed to be possible (as we are restricting the input to algorithms to a finite amount).

The next question to answer is how to construct Gröbner bases to operate upon. Since these are useful for computational purposes, an algorithm for how to compute them will be discussed and proven. Before that, it is important to have an understanding of some key properties we can leverage from this basis.

3.3.1 Properties Of Gröbner Basis

A problem that arises when performing multivariate division is that the order in which the polynomials appear affects the remainder of the quotient. This is particularly difficult when trying to establish deterministic outcomes. By fixing a lexicographical order, Gröbner bases guarantee a unique remainder.

Proposition 3.8. *Let $I \subseteq k[x_1, \ldots, x_n]$ be an ideal and let $G = \{g_1, \ldots, g_t\}$ be a Gröbner basis for I. Then, given $f \in k[x_1, \ldots, x_n]$, there is a unique $r \in k[x_1, \ldots, x_n]$ with the two following properties:*

1. *No term of r is divisible by any of $LT(g_1), ..., LT(g_t)$.*

2. *There is a $g \in I$ such that $f = g + r$.*

Proof. Consider the division algorithm $f = q_1 g_1 \cdots + q_n g_n + r$. Every $LT(g_t)$ is already divided out. In other words, if r had been divisible by the leading term, there would have been a corresponding q_t to engulf it. That proves (1). (2) is solved by considering the fact that we are operating with a Gröbner basis; we must leverage our ability to show that $\langle LT(I) \rangle = \langle LT(g_1), \ldots, LT(g_t) \rangle$. To prove uniqueness, assume that the remainder is non-unique and use Lemma 6 to disprove that assumption. In other words, assume that $f = g + r = g' + r'$ follows our proposition. Rearranging, $r - r' = g - g' \in I$. Now, using the special property of the basis, $LT(r - r') \in \langle LT(I) \rangle = \langle LT(g_1), \ldots, LT(g_t) \rangle$. Since $LT(r - r')$ lies in $LT(g_i)$, it is also divisible by that same $LT(g_i)$. Using the first property, $r - r'$ cannot be divided by $LT(g_i)$, so in order for the division to succeed, $r - r' = 0$. This contradicts the assumption that r was non-unique. □

Another important property is how easily one can verify if a function is in an ideal. The membership problem is solved by dividing f by G and checking if the remainder is zero. If it is, then the function belongs to the ideal. This membership check's concrete nature is useful for computational applications.

3.3.2 Buchberger's Criterion

The following definitions and lemmas are preparatory for algorithmically finding gröbner basis. The algorithm and my implementation using Macaulay2 can be found in section 4.3.

Definition 3.9. We will write $\bar{f}^F$ for the remainder on division of f by the ordered s-tuple $F = (f_1, \ldots, f_s)$. If F is a Gröbner basis for $\langle f_1, \ldots, f_s \rangle$, then we can regard F as a set.

Now that we have a concise way to refer to the remainder of a division by two arbitrary parameters, our discussion around the subject will be less cluttered. A deterministic approach to cancelling leading terms will help in the construction of our algorithm. Below is a definition that introduces the tool we will use to craft the algorithm's kernel.

Definition 3.10. Let $f, g \in k[x_1, \ldots, x_n]$ be nonzero polynomials.

1. If multdeg$(f) = \alpha$ and multdeg$(g) = \beta$, then let $\gamma = (\gamma_1, \ldots, \gamma_n)$, where $\gamma_i = \max(\alpha_i, \beta_i)$ for each i. We call x^γ the least common multiple of $LM(f)$ and $LM(g)$, written $x^\gamma = \text{lcm}(LM(f), LM(g))$.

2. The S-polynomial of f and g is the combination

$$S(f, g) = \frac{x^\gamma}{LT(f)} \cdot f - \frac{x^\gamma}{LT(g)} \cdot g \tag{8}$$

Let us explore the definition provided by the book [DAC10]. Our numerators share the extracted least common multiple. The other interesting part is that the incoming parameterized functions, f and g, are being divided by their own leading term. It seems as though the least common multiple ensures that a cancellation occurs. In other words, this S-polynomial could be an engine for reducing the degree of polynomials. The next lemma showcases an exciting part of mathematics, namely the validation of intuition.

Lemma 3.11. *Suppose we have a sum $\sum_{i=1}^s p_i$ where multdeg$(p_i) = \delta \in \mathbb{Z}_n^{\geq 0}$ for all i. If multdeg$(\sum_{i=1}^s p_i) < \delta$, then $\sum_{i=1}^s p_i$ is a linear combination with coefficients in k, of the S-polynomials $(S(p_j, p_l))$ for $i \leq j, l \leq s$. Furthermore, each $(S(p_j, p_l))$ has multidegree $< \delta$.*

Cox and his book [DAC10] wonderfully turns our attention to an interesting idea. The assumption made in the lemma relies heavily on the properties of the s-polynomial. The written proof exists in the book in chapter 2.6 as Lemma 5. The intuition to extract from the proof can be described as follows.

Consider an addition of polynomials, all having the same multidegree, summing to a lesser multidegree. If you can find an example of that, this lemma ensures that you can rewrite that addition of polynomials as an addition of S-polynomials. Using this fact, the criterion for admission into the Gröbner basis can be formulated.

Theorem 3.12 (Buchberger's Criterion)**.** *Let I be a polynomial ideal. Then a basis $G = g_1, \ldots, g_t$ of I is a Gröbner basis of I if and only if for all pairs $i \neq j$ the remainder, $\overline{S(g_i, g_j)}^{G}$, of division is zero.*

Here, the theoretical implementation is exposed. All one has to do to find a Gröbner basis is to simply check all the g_ts against each other using the constructed s-polynomial. If a nonzero remainder is found, a new g_{t+1} is added to the basis being generated. If the remainder is zero, then we are positive that we have a gröbner basis.

Proof. $\rightarrow$ If G is a Gröbner basis, then we are trying to prove that the remainder $\overline{S(g_i, g_j)}^{G}$ is 0. Leveraging the problem of membership, we can recall that the remainder would be 0 for all members of the Gröbner basis. Since all of the S-polynomials lie in I, the remainder is 0.

$\leftarrow$ Proving the other way is more involved. We start by letting $f \in I$ be nonzero. We will prove that $LT(f) \in \langle LT(g_1), \ldots, LT(g_n) \rangle$. Consider the functions:

$$f = \sum_{i=1}^{t} h_i g_i, h_i \in k[x_1, \ldots, x_n]$$

$$\text{multdeg}(f) = \max\Big\{\text{multdeg}(h_i g_i) \mid h_i g_i \neq 0\Big\}$$

This function f will then be parsed into the following form and minimized with respect to δ: $\delta = \cdots$. The minimization can occur because of our well-ordering. Now this theorem splits into two cases. $\square$

4 Algorithms in Macaulay2

4.1 Exploring Macaulay2

After installing the Windows Subsystem for Linux, the Macaulay2 (M2) playground was ready and open. Since the main book [LM21] I was using had an introductory section to the language, the first order of business was to familiarize myself with EMACS and how the syntax of the language worked. This took quite some time as EMACS/Linux is highly unfamiliar to me and the syntax used when interfacing with the M2 console line is different from when one writes a script.

The very first script I wrote, A.5, mirrors this confusion. Even though I am creating a script, double $*$ signs are being used instead of exponentials because the Swedish keyboard has an odd knack for making the powers small. This led to compiler mismatches and wasted hours. This script is a starting point in how to use the language. Important features such as which ring is being operated upon, which ordering is used, and how to create a Gröbner basis are touched upon. The next couple of book examples, namely A.6 and A.7, show how to construct more involved rings as well as how to evaluate functions in the ring itself. This eased my understanding of how to use rings in the language.

While helpful, these book examples provided little help in how to construct a script that executes any sort of meaningful code. The book displayed clips from the command line of M2, which is not the proper format for writing algorithms. So even though book example A.5 and A.8 show built in ways to perform division and find the remainder, implementing a division algorithm in one and multiple dimensions seemed like an appropriate start to acclimate myself to the language.

4.2 Division Algorithms

Division algorithms have been of interest to mathematicians for a long time. Euclid's algorithm, one of the oldest algorithms created, has applications to division as well. Basic arithmetic with addition, subtraction, and multiplication can be constrained to $\mathbb{Z}$, the integers, while the introduction of division necessitates an entirely different way of interacting with numbers, $\mathbb{Q}$, the rationals. Division is useful, so a mathematical programming language must be equipped with ways to handle it. Macaulay2 has various ways of dividing, but attempting to extend it to divide polynomials left me with a multitude of problems, highlighting how complicated algebra

can actually be.

4.2.1 Univariate Division Algorithm

The first algorithm I tried writing was A.1. This is basic univariate polynomial division, something that most high schoolers learn. The algorithm is rather simple. Take the leading term of the numerator and divide it by the denominator. Add the quotient to the result and subtract the quotient times the denominator from the numerator. Repeat until the numerator is zero or the degree of the denominator exceeds that of the numerator. None of the ordering issues or really anything at all is difficult in this case. The built in division operator works without any issues. Filled with confidence, the multivariate algorithm was attempted.

4.2.2 Multivariate Division Algorithm

The idea behind the multivariate division algorithm is not much different from the univariate one. Divide a polynomial p by an array of polynomials gs. If the leading term of p cannot be divided by the leading term of the first polynomial of gs, move on to the next until one that can is found. If no leading term of gs can divide the leading term of p, then the leading term is subtracted from p and the algorithm marches to the next polynomial. This seems easy enough, but there is quite a lot of complexity hiding between the lines. The first complexity had to do with the fact that there are more dimensions. Dividing with one dimension is easy enough, but when more are introduced, a way to organize the different dimensions is required. MonomialOrdering is introduced to solve this problem.

Another issue was how to divide the leading monomials. With the addition of higher dimensions, the regular division was giving fractions of rings. This is an issue because the algorithm wants to remain within the ring itself, not outside of it. Foolishly, I decided to try and create a leadingTermDivision function that decomposed the polynomials into lists of monomials, coefficients, and exponents. The idea was simple. Macaulay2 generates a list of the exponents, taking the ring elements into account. So if the ring is $Q[x, y, z]$, the polynomial $x^2 * z$ would be decomposed into $\{2, 0, 1\}$. By creating these lists and subtracting the exponents from each other, the division has occurred. All that would be left would be to recreate the polynomial using the generators of the ring and raise them to the exponent in the listed order. However, here my lacking understanding of Macaulay2 and, more generally, ring algebra can be seen. The generators of the ring and the ring itself

are different. In line 63 of A.2, toList gens ring p is used. Here, we find the ring of the polynomial p, find the generators of the ring, and then create a list of them. However, when later applying the exponential list to the generators, the returned polynomial is not an element of the ring that the function caller is using; it simply has the same variables.

Instead of recreating a way to divide monomials or using fractions of rings, there is a division that ensures the result remains in the ring. It does so by using division with remainder. With this realization, the algorithm was ready to be implemented. By first checking whether the modulo of the leading monomial would be 0, we can ensure that the ring division would not produce a remainder. So if the modulo did produce a remainder, the next gs was considered. If there was no remainder, essentially the univariate case was established.

One important note here is that, as another reason to use Gröbner bases, the order of the polynomials sent in, gs, matters significantly. Different orderings of the gs list generate different results. A test showing this can be found in A.2. Gröbner bases eliminate this reliance on the order, which is desirable.

4.3 The Buchberger Algorithm

Buchberger's algorithm is the first algorithm that properly involves a Gröbner basis. It also immediately solves the issue faced by the multivariate division algorithm. It primarily does so by dividing by an ideal instead of a set of polynomials. This set must be "special" in the sense that the remainder of the division is also special. In fact, the remainder of the generators must be uniquely determined, and if the remainder is zero, then it is equivalent to having membership in the ideal I [LM21]. As it turns out, Gröbner bases have these two properties.

In order to divide by the Gröbner basis (or that "special" ideal), it must first be constructed. Luckily, the Buchberger algorithm does just that. The full implementation can be found in A.3, but there are interesting mathematical properties to discuss in this algorithm.

4.3.1 Mathematical properties

The most interesting part of this algorithm is the S-polynomial, which is specifically designed to cancel the leading terms of the two polynomials being compared [LM21]. To find this polynomial, the lead monomial of each polynomial is extracted, and then

the greatest exponent of each variable is found. This is equivalent to finding the least common multiple between the monomials, and this is how it is implemented in A.3. Then, the lead term of each polynomial divides the least common multiple, and the entire fraction multiplies the original polynomial. This is repeated for the other polynomial, and they are subtracted from each other. Then the S-polynomial has been found.

The use of this S-polynomial is a bit more involved. The Buchberger criterion states that a basis $G = \{g_1, \ldots, g_n\}$ is a Gröbner basis if and only if, for all pairs $i \neq j$, the remainder on division of $S(g_i, g_j)$ by G is 0 [DAC10]. In other words, the algorithm has to generate pairs of all polynomials, find the S-polynomial, and then ensure that the remainder after doing multivariate polynomial division is 0. If it is not, the remainder is appended and the algorithm continues until the condition that the remainder is 0 for all pairs is met.

4.4 Solving Systems of Polynomials

The polynomial solution was by far the most involved algorithm to construct from a Macaulay2 standpoint. At this point, I was feeling much more comfortable with the language, the syntax no longer holding me back, and a more functional approach being taken. I started by following examples from the book [LM21], namely A.11 and A.12.

4.4.1 Book Examples of Polynomial Solutions

To start, a single-variable polynomial is considered. The quotient ring is generated, and the monomial basis is found. This is surprisingly easy in Macaulay2, and most of these complex operations are simply done under the hood. One important note is that the basis function of Macaulay2 generates elements outside the ring, so a nifty "lift" is used to bring them back in. Consider the fact that $\frac{4}{2}$ can be "lifted" from $\mathbb{Q}$ to $\mathbb{Z}$. After this, we find the $\mathbb{Q}$ matrix associated with $\mathbb{Z}$, or M_z^Q. While perhaps tedious to do by hand, taking the modulo by the ideal is easy in Macaulay2. Building the coefficient matrix is similarly easy, as we simply extract the coefficients of the remainder of the modulo with the ideal. All that remains is to raise the matrix to the necessary powers to reconstruct the polynomial of interest, and then one is done.

The second book example, A.12, complicates things slightly by introducing a

second variable. However, at this point I realized that a dedicated function for solving M^Q would be necessary. All that needed to be done was to extract the logic used in A.11 and refactor it into its own function. After testing to ensure that they returned the same result, implementing A.12 was easy enough. Solving M^Q is as easy as sending the ring, ideal, and the monomial of interest for the matrix. Solving the eigenvalues is a piece of cake as well, with the built-in functionality. Filled to the brim with confidence, the main algorithm was attempted.

4.4.2 Solving a System of Polynomial Equations

This was the final test for everything I had learned in Macaulay2. Equipped with the function to solve M^Q, I thought it would be easy. Starting out, for a ring $R[x_1, \ldots, x_n]$, all the $M^Q_{x_i}$ had to be found. This was easy enough, and finding the eigenvalues thereafter was also something previously done in the book exercises. However, the difficult part came when having to construct the candidate points to plug back into the initial ideal. Using "fold", which is the easiest way to construct a composite function, and "table", which takes two lists and creates all possible pairs from them, these in conjunction create all possible sets of the eigenlists. I am immensely proud of this, and it shows how much I have developed with this language. However, when I evaluate the candidate points over the ideal to see which give a solution to 0, a slight problem occurs. Since the ring is in $\mathbb{R}$, the computations are not exact, and slight variations cause duplicates. Even adding an epsilon of $1e^{-6}$ in order to check whether there are duplicate solutions did not help. So, in order to solve this issue, there is an additional "unique solutions check". Here the check is to ensure that the sum of the distances between all the points is less than the epsilon.

A Macaulay2 source files

A link to all the source files can be found at the github link [Eri26] under the m2 folder.

A.1 Algorithm121.m2

```
univariatePolynomialDivision = (p,g) -> (
    q := 0;
    r := p;
    while (r != 0_R and degree(g) < degree(r)) do (
        t := leadTerm(r) / leadTerm(g);
        q = q+t;
        r = r-t*g;
        );
    (q,r)
    )

runUnivariateTest = () -> (
    R = QQ[x];
    p := x^6 - 1;
    g := x^4 - 1;

    (resultQ, resultR) := univariatePolynomialDivision(p,g);

    print(resultQ == x^2);
    print(resultR == x^2 - 1);
)
```

Listing 1: Macaulay2 script Algorithm121.m2

A.2 algorithm123.m2

```
multivariatePolynomialDivision = (p, gs) -> (
    r := 0_(ring p);
    f := p;
    qs := for i from 0 to #gs-1 list 0_(ring p); --generating
        dummy variable for result

    while f != 0_(ring p) do (
        i := 0;
```

```
8           divisionOccured := false;
9           --print "Outer  loop";
10
11          while (i < #gs and divisionOccured == false) do (
12              --print "Inner loop";
13              lmf := leadMonomial f;
14              lmg := leadMonomial (gs#i);
15
16              if (lmf % lmg == 0) then (
17                  monomialQuotient := lmf // lmg;
18
19                  coefficientQuotient := leadCoefficient f /
                        leadCoefficient gs#i;
20
21                  t := coefficientQuotient * monomialQuotient;
22
23                  qs = replace(i, qs#i + t, qs);
24                  f = f - t*gs#i;
25
26                  divisionOccured = true;
27                  --print "we were divisible";
28              )
29              else (
30                  i = i+1;
31
32                  --print "not divisible, function further along";
33              )
34          );
35
36          if divisionOccured == false then (
37              r = r + leadTerm(f);
38              f = f- leadTerm(f);
39             --print "division did not occur, subtracting leading
                  term and continuing";
40          );
41      );
42      return (qs, r)
43  )
44
45  -- This below code is useless, it was an attempt to get around the
        fact that i couldnt devide with the leadTerm.
```

```
-- However, I needed to seperate out the divisions as well as use "//" instead of "/"
leadingTermDevision = (p,g) -> (
    lmP := leadMonomial(p);
    lmG := leadMonomial(g);
    lcP := leadCoefficient(p);
    lcG := leadCoefficient(g);
    exponentsP := flatten exponents lmP;
    exponentsG := flatten exponents lmG;

    divisible := all(#exponentsP, i -> exponentsP#i >= exponentsG#i); --checking that all exponents are greater or equal
    result := 0_(ring p);

    if divisible then(
        coefficientResult := lcP / lcG;

        exponentsQ := for i from 0 to (#exponentsP-1) list (exponentsP#i - exponentsG#i); --doing the division

        ringVariables := toList gens ring p;

        Q := product(#ringVariables, i -> ringVariables#i ^ (exponentsQ#i)); --generating the the polynomial

        result = coefficientResult * Q;
     );

    return (divisible, result);
 )

 runMultimultivariateTest = () -> (
     R = QQ[x,y,MonomialOrder=>Lex];
     p := -x^2*y-x^2+x*y+y^3;
     g1 := x*y-x+y-1;
     g2 := x+y+1;

     (resultG1, resultRemainder1) := multivariatePolynomialDivision(p,{g1,g2});
     (resultG2, resultRemainder2) := multivariatePolynomialDivision(p,{g2,g1});
```

```
81
82      print "-----------{g1,g2}------------";
83      print resultG1;
84      print resultRemainder1;
85      print (resultG1#0 == -x+4);
86      print (resultG1#1 == -2*x+5);
87      print (resultRemainder1 == y^3-9*y-1);
88
89      print "-----------{g2,g1}------------";
90      print resultG2;
91      print resultRemainder2;
92      print (resultG2#1 == 0);
93      print (resultG2#0 == -x*y-x+y^2+3*y+1);
94      print (resultRemainder2 == -4*y^2-4*y-1);
95  )
```

Listing 2: Macaulay2 script algorithm123.m2

A.3 algorithm124.m2

```
1   load "algorithm123.m2";
2
3   buchberger = ps -> (
4       g := ps;
5       gHat := {0_(ring  ps#0)};
6
7       while g != gHat do (
8           gHat = g;
9
10          -- generating all the possible pairs by looping through i
                and then from j -> end
11          polynomialPairs := flatten for i from 0 to #gHat-1 list
                for j from i+1 to #gHat-1 list (gHat#i, gHat#j);
12
13          i := 0;
14          while i < #polynomialPairs do (
15              p := (polynomialPairs#i)#0;
16              q := (polynomialPairs#i)#1;
17
18              xDelta := lcm(leadMonomial(p), leadMonomial(q));
19
```

```
            s := xDelta // leadTerm(p) * p - xDelta // leadTerm(q)
                * q;

            (qs, r) = multivariatePolynomialDivision(s, gHat);

            if r != 0_(ring ps#0) then (
                g = append(gHat,r);
                );

            i = i+1;
            );
        );
    print g;
    return g;
    )

testAlgorithm = () -> (
    R = QQ[x,y, MonomialOrder => GLex];

    p1 := x^2 + y;
    p2 := x^4 + 2*x^2*y + y^2 + 3;

    myBasis := gb ideal (buchberger({p1, p2}));
    correctBasis := gb ideal(p1, p2);
    areTheyEqual := myBasis == correctBasis;

    print "My basis:";
    print myBasis;

    print "Built-in basis:";
    print gens correctBasis;

    print "Are they equal?";
    print areTheyEqual;
    )

--again, this is not needed and M2 implements LCM.
findXDelta = (p,q) -> (
    alpha := flatten exponents leadMonomial(p);
    gamma := flatten exponents leadMonomial(q);

```

```
60      delta := for i from 0 to (#alpha-1) list
            (max(alpha#i,gamma#i)); --getting the max of each variables
            exponent
61
62      ringVariables := toList gens ring p;
63      xDelta := product(#ringVariables, i -> ringVariables#i ^
            (delta#i)); -- reconstructing the variable
64
65      return xDelta
66      )
67
68  testGamma = () -> (
69      R := RR[x,y, MonomialOrder => GLex];
70      p := x^3*y^2-x^2*y^3+x;
71      q := 3*x^4*y +y^2;
72
73      xDelta := findXDelta(p,q);
74      xDeltaShouldBe := x^4*y^2;
75
76      print "-----------gamma results------------";
77      print xDelta;
78      print xDeltaShouldBe;
79      print("the result is " | toString(xDelta == xDeltaShouldBe))
80      )
```

Listing 3: Macaulay2 script algorithm124.m2

A.4 algorithm161.m2

```
1   solveSystemOfPolynomialEquations = (theRing, theIdeal) -> (
2       varsToLoopOver := gens theRing;
3
4      -- get the list of eigenvalues
5       eigenList := for var in varsToLoopOver list (
6           qMatrix := solveQMatrix(theRing, theIdeal, var);
7
8            --ensure that the eigenvector can be solved and save it
                 to the list
9            toList eigenvalues(sub(qMatrix,CC))
10          );
11
```

```
12      -- use fold to go through all elements in the eigenList
            outwards
13      candidatePoints := fold((L1, L2) -> (
14              --use table to create a table of all the pairs and
                    flatten down
15              flatten table(L1, L2, (a, b) -> (
16                      --if we are past first instance, we append
                            instead of create a list
17                      if instance(a, List) then a | {b} else {a, b}
18                      )
19                  )
20              ),
21          eigenList);
22
23      epsilon := 1e-6;
24      functionList := flatten entries gens theIdeal;
25
26      validSolutions := select(candidatePoints, pt -> (
27              -- ensure that x_n maps to the the nth point
28              subMap = apply(#varsToLoopOver, i -> varsToLoopOver#i
                    => pt#i);
29              -- sub in the values
30              all(functionList, p -> abs(sub(p, subMap)) < epsilon)
31              ));
32
33      -- something is wrong with my solveQMatrix which results in
            cluttered returns.
34      -- so for now just to check if it works, we remove the
            duplicates
35      uniqueSolutions := {};
36      scan(validSolutions, v -> (
37              if not any(uniqueSolutions, u -> (
38                      dist := sqrt(sum(0..#v-1, i -> abs(v#i -
                            u#i)^2));
39                      dist < epsilon
40                      )) then uniqueSolutions =
                            append(uniqueSolutions, v);
41              ));
42
43      uniqueSolutions
44      )
45
```

```
46
47 testAlgorithm2D = () -> (
48     R := CC[x, y];
49     I := ideal(x^2 + y^2 - 5, x*y - 2);
50
51     myResults := solveSystemOfPolynomialEquations(R, I);
52     solution := {{-2, -1}, {-1, -2}, {2, 1}, {1, 2}};
53
54     print ("The 2D test output: " | toString  myResults);
55
56     print ("Verified solution: " | toString solution);
57 )
58
59 testAlgorithm4D = () -> (
60     R4 := CC[x1, x2, x3, x4];
61
62     p1 := x1^2 + x2^2 - 2;
63     p2 := x1 - x2;
64     p3 := x3^2 + x4^2 - 13;
65     p4 := x3*x4 - 6;
66     I4 := ideal(p1, p2, p3, p4);
67
68     results4D := solveSystemOfPolynomialEquations(R4, I4);
69
70     if #results4D == 8 then (
71         print "SUCCESS: Found all 8 points in 4D space.";
72     ) else (
73         print ("FAILURE: Expected 8 points, but found " |
            #results4D);
74     );
75 )
76
77 solveQMatrix = (theRing, theIdeal, thePolynomial) ->(
78     q := theRing / theIdeal;
79
80     basisList := flatten entries basis q;
81     polyBasis := for b in basisList list lift(b,theRing); --basis
        to R
82
83     rows := for b in polyBasis list (
84         currentPoly := thePolynomial * b;
85         r := currentPoly % theIdeal;
```

```
86
87          coeffsMatrix := last coefficients(r, Monomials =>
                polyBasis);
88          flatten entries coeffsMatrix
89          );
90
91      return transpose matrix rows;
92      )
93
94  print "-------2D CASE--------";
95  testAlgorithm2D();
96  print "------4D CASE---------";
97  testAlgorithm4D();
98  print "-------DONE--------";
```

Listing 4: Macaulay2 script algorithm161.m2

A.5 BookExample2.1.1.m2

```
1  R = QQ[x_1..x_4, MonomialOrder => Lex]
2
3  p1 = x_1-x_2**2+x_3
4
5  p2 = x_1**2+x_2*x_3**2+x_4
6
7  I = ideal(p1,p2)
8
9  G = gens gb I
10
11 polyToDivide = x_1**2+x_2**2+x_3**2+x_4**2-1
12
13 rem = polyToDivide%I
14
15 print("the rem is: ")
16 print(rem)
```

Listing 5: Macaulay2 script BookExample2.1.1.m2

A.6 BookExample2.2.m2

```
1  R = QQ[x_1,x_2,x_3]
```

```
2
3 p = (1/2)+x_1^2*x_2*x_3^4+(6/5)*x_1*x_3+2
4
5 -- output is a polynomial in R
```

Listing 6: Macaulay2 script BookExample2.2.m2

A.7 BookExample222.m2

```
1 R = frac(QQ[a]/ideal(a^2-2))[x_1,x_2,x_3]
2
3 substitute(a^2, R)
4 -- output is that a^2 = 2
5
6 p = x_1 + x_2 + a*x_3
```

Listing 7: Macaulay2 script BookExample222.m2

A.8 BookExample231.m2

```
1 R = QQ[z]
2
3 p = z^4+3*z^3-3*z^2+3*z-4
4
5 pc = -2*z^4+3*z^3-6*z^2+5*z-4
6
7 g = z^2+1
8
9 (q,r)= quotientRemainder(p,g)
10
11 (qc, rc) = quotientRemainder(pc,g)
```

Listing 8: Macaulay2 script BookExample231.m2

A.9 BookExample232.m2

```
1 R=QQ[c_1,c_2,c_3,c_4, MonomialOrder=>Lex]
2 M = matrix{{1,-2,0,-1},{4,0,7,3}, {7,-6,7,0}}
3 C = matrix{{c_1},{c_2},{c_3},{c_4}}
4 I = ideal(M*C)
```

```
GI = value toString gens gb I
(Ce,Me) = value toString coefficients(GI,
    Monomials=>{c_1,c_2,c_3,c_4})

Mr = value toString transpose Me

--powerful way to solve the row reduced form
inverse(substitute(submatrix(Mr, {0,1}),QQ))*Mr
```

Listing 9: Macaulay2 script BookExample232.m2

A.10 documentationConditionals.m2

```
isEven = i -> i % 2 == 0

oddOrEven = i -> (
    if isEven i then "even"
    else "odd")
```

Listing 10: Macaulay2 script documentationConditionals.m2

A.11 Example1612.m2

```
runExample = () -> (
    R = RR[z];

    g := z^3-3*z^2+2*z;

    I = ideal g;

    q := R / I;

    basisList := flatten entries basis q;
    polyBasis := for b in basisList list lift(b, R); -- ensuring
        that the actualy polynomials exists in R and not only in q

    rows := for b in  basisList list (
        currentPoly = z *lift(b,R);

        r := currentPoly % I;

```

```
18          --taking #1 here since coefficients returns a bundle of
                want the coefficients, no the monomials
19          coeffsMatrix := (coefficients(r, Monomials =>
                polyBasis))#1;
20          flatten entries coeffsMatrix
21          );
22
23      zmatrix := matrix rows;
24
25      finalZMatrix := zmatrix^2+zmatrix+id_(target zmatrix);
26      return finalZMatrix;
27  )
28
29
30   runExampleWithFunction = () -> (
31      R = RR[z];
32
33      g := z^3-3*z^2+2*z;
34
35      I = ideal g;
36
37      load "algorithm161.m2";
38
39      zmatrix := solveQMatrix(R, I, z);
40
41      finalZMatrix := zmatrix^2+zmatrix+id_(target zmatrix);
42      return finalZMatrix;
43  )
44
45  testFunction = () -> (
46      preBuiltMatrix := runExample();
47      matrixFromFunction := runExampleWithFunction();
48
49      print "Matrix from the example";
50      print preBuiltMatrix;
51
52      print "Matrix using refactored function";
53      print matrixFromFunction;
54   )
```

Listing 11: Macaulay2 script Example1612.m2

A.12 Example1613.m2

```
runExample = () -> (
    load "algorithm161.m2";

    R = RR[x,y];
    f := (y^2+x-3, x^2-4*x*y-y^2-10*y+15);
    I := ideal f;

    regMatrix := solveQMatrix(R,I,1);
    xMatrix := solveQMatrix(R,I,x);
    xyMatrix := solveQMatrix(R,I,x*y);
    yMatrix := solveQMatrix(R,I,y);

    --only care about the x,y individual ones
    xEigens := (eigenvectors sub(xMatrix,CC))#0;
    yEigens := (eigenvectors sub(yMatrix,CC))#0;

    varietyList := for i from 0 to  #xEigens-1 list {xEigens#i,
       yEigens#i};
    print varietyList;
    )
```

Listing 12: Macaulay2 script Example1613.m2

References

[DAC10] Donal B O'Shea David A Cox, John Little. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Springer Publishing Company, third edition, November 2010.

[DF03] David S. Dummit and Richard M. Foote. *Abstract Algebra*. Wiley, Hoboken, NJ, USA, 3 edition, 2003.

[Diand] Jaime Dianzon. Keeping it real. GeoGebra interactive construction, n.d. Available at: https://www.geogebra.org/m/n8hauvp6.

[Eri26] Christian Eriksson. M2 algorithms. Available at https://github.com/Skalmanen/MM6010, 2026.

[Gal25] Jean H. Gallier. Basics of affine geometry. In Jean H. Gallier, editor, *Geometric Methods and Applications for Computer Science and Engineering*, pages 7–63. Springer, 2025. Chapter 2 from online course materials. URL: https://www.cis.upenn.edu/~cis6100/geombchap2.pdf.

[Lat] Lattice and boolean algebra 2.1 algebra 2.2 lattice. URL: https://api.semanticscholar.org/CorpusID:18433539.

[LM21] Antonio Tornambè Laura Menini, Corrado Possieri. *Algebraic Geometry for Robotics and Control Theory*. WORLD SCIENTIFIC (EUROPE), 2021. arXiv:https://www.worldscientific.com/doi/pdf/10.1142/q0308.

[Wei70] Louis M. Weiner. *Introduction to Modern Algebra*. Harcourt, Brace & World, New York, NY, USA, 1970.